



A SQL Query is Just a DAG

Building a SQL Engine on Apache Airflow



Hussein Awala • Datadog • Apache Airflow PMC • Airflow Summit 2026

Hi, I'm Hussein



- ▶ Senior Data Engineer at Datadog
- ▶ PMC member and committer on Apache Airflow
- ▶ I like opening systems to see how they work inside

This talk is a side project that started as a question I could not stop thinking about.

Where this idea came from

- ▶ I help maintain Airflow, and I keep seeing the same thing
- ▶ People do heavy data work inside tasks — not just scheduling
- ▶ And Airflow already sends that work to Celery or Kubernetes



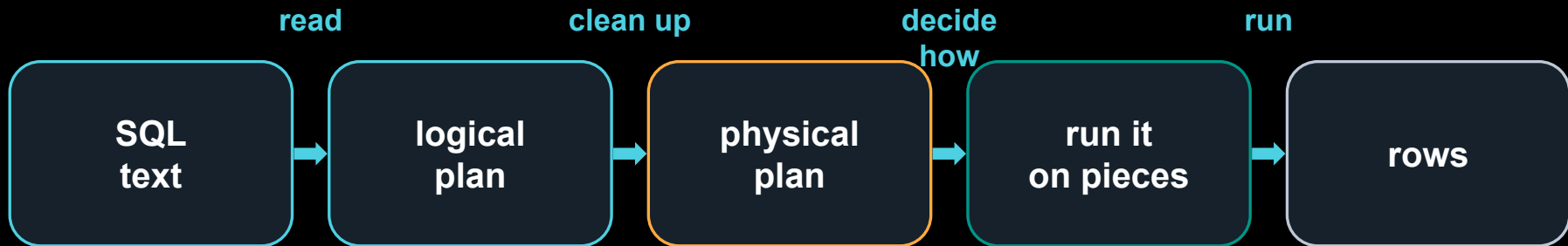
Same shape. So what is missing for Airflow to be the scheduler of a SQL engine?

What happens after you hit run?

- ▶ You write `SELECT ... GROUP BY`
- ▶ You get rows back
- ▶ In the middle there is a box nobody opens
- ▶ Today we open it

Three words to keep: partition · shuffle · pipeline-breaker

What any SQL engine does



- ▶ Logical plan — a tree that says what to compute
- ▶ Physical plan — how many pieces, and how data moves
- ▶ That second one already looks like a graph. A DAG.

Three words that explain parallel work

PARTITION

One slice you handle alone

more slices = more at once

SHUFFLE

Move rows by key so they meet

keeps GROUP BY / JOIN right

PIPELINE-BREAKER

Must see its input first

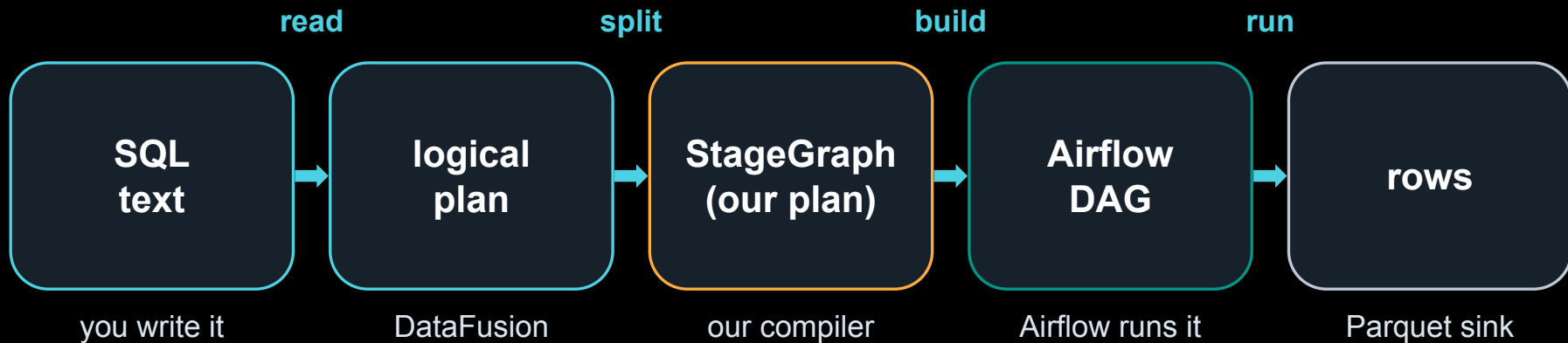
group by · join · sort



The plan is a DAG. Airflow runs DAGs.

So let Airflow be the scheduler, and let DataFusion do the math inside each task.

The mapping – what I used for each step



Between two stages, data moves as Parquet files on shared storage.

Step 1 – SQL to a logical plan

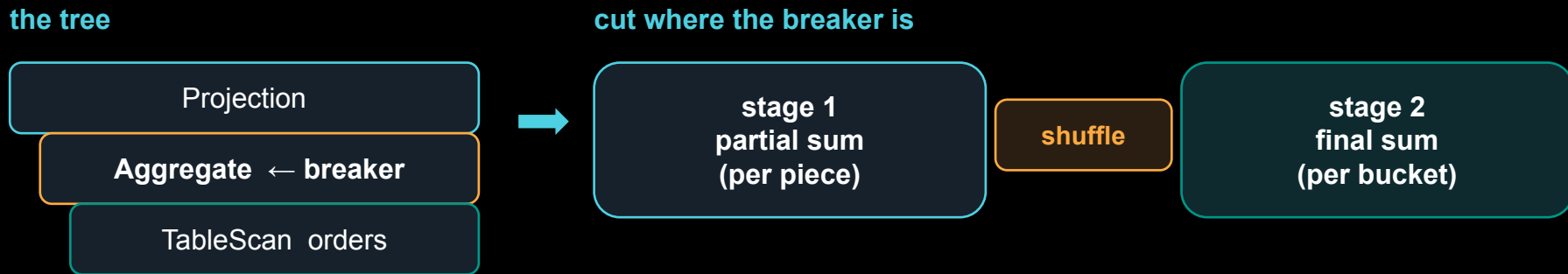
- ▶ DataFusion reads and checks the SQL
- ▶ We give it the columns, not the data
- ▶ A wrong column fails now, not mid-run
- ▶ We read the tree; we wrote no parser

```
ctx = datafusion.SessionContext()  
plan = ctx.sql(sql).logical_plan()
```



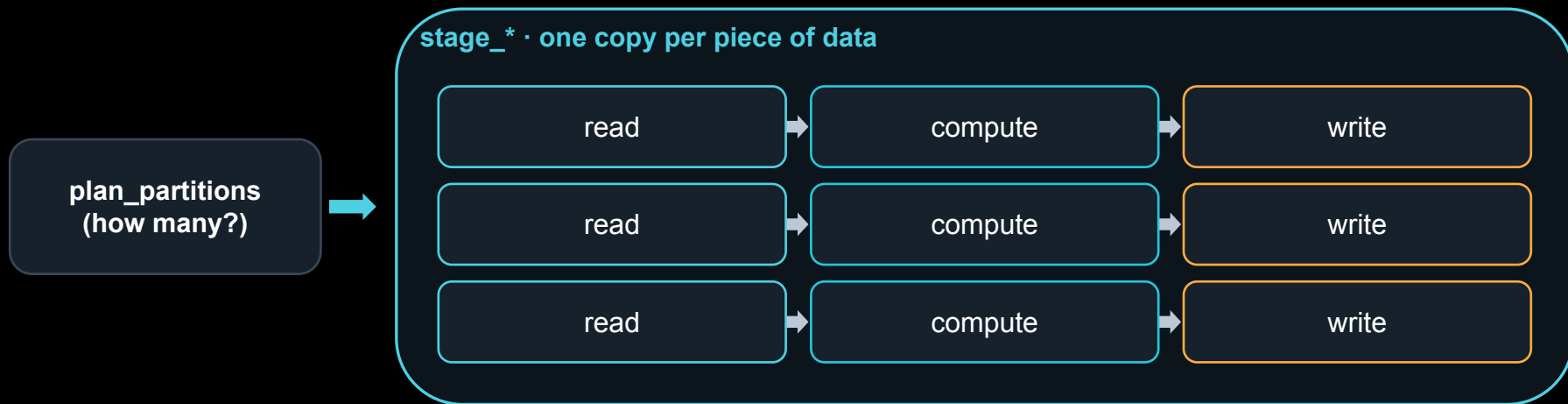
the tree we walk in `sql_compiler.py`

Step 2 – cut the plan into stages



- ▶ A new stage every time we hit a breaker
- ▶ GROUP BY becomes partial sum, then final sum — sum of sums
- ▶ Average does not work that way, so we say no instead of guessing

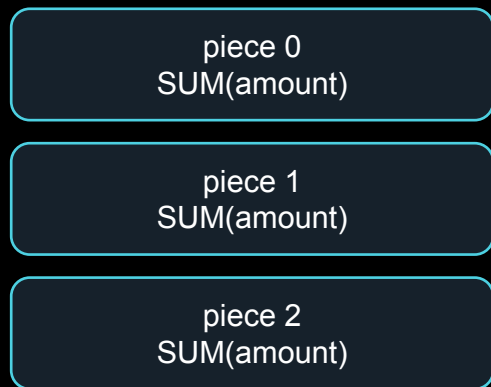
Step 3 – one stage is one task group



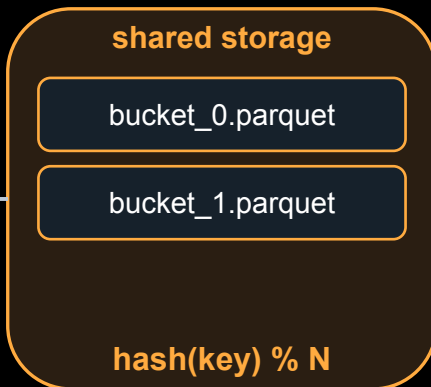
Airflow copies the group once per piece of data — that is the parallel part.

The shuffle: tasks cannot talk, so they write it down

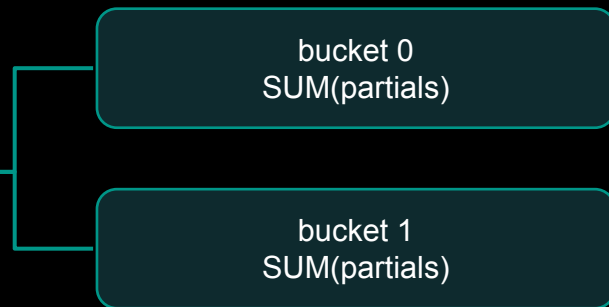
STAGE 1 · partial



SHUFFLE



STAGE 2 · final



= total per key



Let's look at real code, then watch it run

First the file you would actually write. Then a recording of the DAG running.

Real data, first query: filter

This is the whole file. No test data, no temp folders.

1 the files and the query

```
DAG_ID = "sql_as_dag_orders"

INPUT_URIS = [
    "s3://bucket/orders/dt=2026-08-01/",
    "s3://bucket/orders/dt=2026-08-02/",
]
OUT_URI = "s3://bucket/out/orders/"

_SQL = """
SELECT customer_id, amount
FROM orders
WHERE amount > 25
"""
```



2 the source, then build the DAG

```
source = Source(
    source_id="orders_src",
    table_name="orders",
    connector="parquet",
    options={"uris": INPUT_URIS},
)

graph = compile_sql(
    _SQL, [source],
    stage_id_prefix="orders",
)

dag = dag_from_stages(
    graph,
    dag_id=DAG_ID,
    sink=Sink("parquet",
        {"base_uri": OUT_URI}),
    schedule=None)
```

Local paths work the same way — everything goes through Airflow's ObjectStoragePath.

Same file, now a GROUP BY

Only the query changes — and one setting for the number of buckets.

1 the files and the query

```
DAG_ID = "sql_as_dag_groupby"
_NUM_BUCKETS = 4

INPUT_URIS = [
    "s3://bucket/orders/dt=2026-08-01/",
    "s3://bucket/orders/dt=2026-08-02/",
]
OUT_URI = "s3://bucket/out/total/"

_SQL = """
SELECT customer_id,
       SUM(amount) AS total
FROM orders
GROUP BY customer_id
"""
```



2 the source, then build the DAG

```
source = Source(
    source_id="orders_src",
    table_name="orders",
    connector="parquet",
    options={"uris": INPUT_URIS},
)

graph = compile_sql(
    _SQL, [source],
    num_buckets=_NUM_BUCKETS,
    stage_id_prefix="orders",
)

dag = dag_from_stages(
    graph,
    dag_id=DAG_ID,
    sink=Sink("parquet",
              {"base_uri": OUT_URI}))
```

Or pass a BucketingPolicy instead, and a task picks the number of buckets at run time.

Being honest: this is a teaching engine

- ▶ It works, and it gives correct answers on a real part of SQL
- ▶ It is not fast, and it is not trying to be
- ▶ Every stage writes files and reads them back
- ▶ One task per file, and Airflow stops around a thousand

The idea is real. Airflow is just not built to run huge jobs like this — and that is fine.

Airflow is general – and it keeps growing

- ▶ Today: a DAG is a file, parsed in a loop, run on a schedule
- ▶ A query wants: send it once, run it once, gone
- ▶ One-shot DAG runs through the API — Airflow 3 moves that way
- ▶ AIP-113 dynamic task groups → change the plan while it runs (Spark's AQE)
- ▶ AIP-111 task loops → queries that repeat until they settle (graphs)

Both are proposals. The point: every new Airflow feature gives the engine something new.

What to take home

- ▶ A SQL engine is a planner plus workers over pieces of data
- ▶ Partition, shuffle, pipeline-breaker — you can name them now
- ▶ You don't build an engine from zero; you put pieces together
- ▶ How far you can go depends on the scheduler you pick

Thank you – and where to look

BOOK How Query Engines Work — Andy Grove · howqueryengineswork.com

BUILT ON Apache Airflow · Apache DataFusion · Apache Arrow · Parquet

CODE [sql-as-dag](https://github.com/hussein-awala/sql-as-dag) · github.com/hussein-awala/sql-as-dag

IN THE REPO the demo DAGs, more examples, and diagrams of how it works

ME Hussein Awala · Datadog · Apache Airflow PMC

Questions?



the repo